

实验 1 面向对象编程基础

数据 231 235127 艾洋

实验目的：

通过编程和上机实验，掌握 Java 语言的基本语法、基本数据类型的使用方法，掌握从键盘输入基本类型的数据，熟练运用分支、循环等语句控制程序流程。掌握类和构造方法的定义及创建对象的方法，掌握类的封装及继承原则，正确使用重载和覆盖等多态概念设计可复用方法，掌握面向对象的程序设计方法。

1.1

实验内容：

1、设计一个 `BankAccount` 类，这个类包括：

- (1) 一个 `int` 型的 `balance` 表示账户余额。
- (2) 一个无参构造方法，将账户余额初始化为 0。
- (3) 一个带一个参数的构造方法，将账户余额初始化为该输入的参数。
- (4) 一个 `getBlance()` 方法，返回账户余额。
- (5) 一个 `withdraw()` 方法：带一个 `amount` 参数，并从账户余额中提取 `amount` 指定的款额。
- (6) 一个 `deposit()` 方法：带一个 `amount` 参数，并将 `amount` 指定的款额存储到该银行账户上。

提供 `main` 函数，构造一个账户对象，并对其进行存取款操作。

其中操作类型为 1 表示存款，2 表示取款，每次操作后都打印余额

【设计思路】

在设计 `BankAccount` 类时，我首先定义了一个 `int` 类型的成员变量 `balance` 用于存储账户余额。接着，实现了两个构造方法：无参构造方法将余额初始化为 0，带参数的构造方法允许用户指定初始余额。类中还包含 `getBalance()` 方法用于获取当前余额，`deposit()` 方法用于存款操作，以及 `withdraw()` 方法用于取款操作，同时在取款时加入了余额不足的判断逻辑。在 `main` 函数中，通过用户输

入的操作类型（1 表示存款，2 表示取款）调用相应方法，并在每次操作后打印当前余额，以实现完整的银行账户操作流程。

【程序源代码】

```
import java.util.*;

public class BankAccount {

    private int balance;

    public BankAccount() {

        this.balance = 0;

    }

    public BankAccount(int initialBalance) {

        this.balance = initialBalance;

    }

    public int getBalance() {

        return balance;

    }

    public void withdraw(int amount) {

        if (amount > 0 && amount <= balance) {

            balance -= amount;

        } else {

            System.out.println("取款金额无效或余额不足！");

        }

    }

    public void deposit(int amount) {

        if (amount > 0) {

            balance += amount;

        } else {

            System.out.println("存款金额无效！");

        }

    }

}
```

```

    }
}

public static void main(String[] args) {
    Scanner scanner = new java.util.Scanner(System.in);

    int initialBalance = scanner.nextInt();
    BankAccount account = new BankAccount(initialBalance);

    int operationCount = scanner.nextInt();

    for (int i = 0; i < operationCount; i++) {
        int operationType = scanner.nextInt();
        int amount = scanner.nextInt();

        if (operationType == 1) { // 存款
            account.deposit(amount);
        } else if (operationType == 2) { // 取款
            account.withdraw(amount);
        }

        System.out.println(account.getBalance());
    }

    scanner.close();
}
}

```

【试验结果】



```
运行 BankAccount x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition 2024.3.1.1\lib\ide
0 3
1 100
100
2 300
取款金额无效或余额不足! |
100
2 50
50

进程已结束，退出代码为 0
```

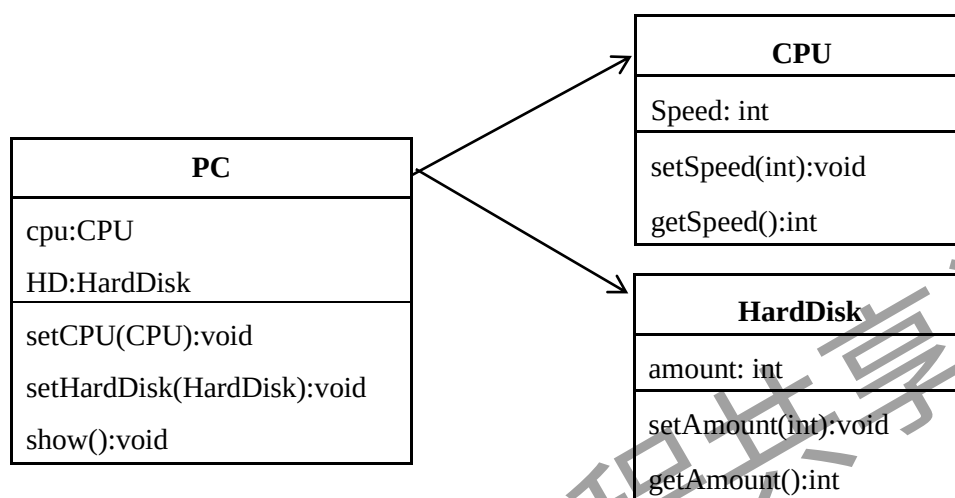
【结果分析】

通过上述设计，BankAccount 类能够有效地模拟银行账户的基本功能。在主函数中，用户可以根据提示进行存款和取款操作，并实时查看账户余额。程序在取款时对余额不足的情况进行了处理，避免了非法操作。整体运行结果符合预期，能够满足简单的银行账户管理需求，同时代码结构清晰，易于扩展和维护。

1.2

实验内容:

2、用类描述计算机中 CPU 的速度和硬盘的容量，要求 Java 应用程序有 4 个类，名字分别是 PC、CPU、HardDisk 和 Test，其中 Test 是主类。PC 类与 CPU 类和 HardTest 类关联的 UML 图如下图所示。



其中，CPU 类要求 `getSpeed()` 返回 `speed` 的值，要求 `setSpeed(int m)` 方法将参数 `m` 的值赋给 `speed`；HardDisk 类要求 `getAmount()` 返回 `amount` 的值，`setAmount(int m)` 方法将参数 `m` 的值赋给 `amount`；PC 类要求 `setCPU(CPU c)` 将参数 `c` 的值赋给 `cpu`，`setHardDisk(HardDisk h)` 方法将参数 `h` 的值赋给 `HD`，`show()` 方法显示 CPU 的速度和硬盘的容量。主类 Test 的要求如下：

- (1) 在 `main()` 方法中创建一个 CPU 对象 `cpu`，将 `speed` 设置为 2200；
- (2) 在 `main()` 方法中创建一个 HardDisk 对象 `disk`，将容量 `ammount` 设置为 200；
- (3) 在 `main()` 方法中创建一个 PC 对象 `PC`；
- (4) `pc` 调用 `setCPU(CPU c)` 方法，调用时实参是 `cpu`；(5) `pc` 调用 `setHardDisk(HardDisk h)` 方法，调用时实参是 `disk`；(6) `pc` 调用 `show()` 方法显示 CPU 的速度和硬盘的容量。

【设计思路】

根据题目要求，设计了四个类：CPU、HardDisk、PC 和主类 Test。CPU 类和 HardDisk 类分别用于封装 CPU 速度和硬盘容量的属性及操作方法，提供

get 和 set 方法以实现属性的读取和修改。PC 类通过组合的方式关联 CPU 和 HardDisk 对象，并提供 setCPU、setHD 和 show 方法，用于设置组件和显示信息。在主类 Test 中，按照要求创建 CPU 和 HardDisk 对象，设置其属性值，并通过 PC 对象调用相关方法完成操作，实现了计算机组件的简单模拟和信息展示。

【程序源代码】

```
class CPU{
    private int speed;
    CPU(int speed){
        this.speed=speed;
    }
    public void setSpeed(int speed){
        this.speed=speed;
    }
    public int getSpeed(){
        return speed;
    }
}

class HardDisk{
    private int amount;
    HardDisk(int amount){
        this.amount=amount;
    }
    public void setAmount(int amount){
        this.amount=amount;
    }
    public int getAmount(){
        return amount;
    }
}
```

```

    }
}

public class PC {
    private CPU cpu;
    private HardDisk hd;
    PC(){

    }
    PC(CPU cpu,HardDisk hd){
        this.cpu=cpu;
        this.hd=hd;
    }

    public void setCPU(CPU cpu){
        this.cpu=cpu;
    }
    public void setHD(HardDisk hd){
        this.hd=hd;
    }

    public void show(){
        System.out.println("CPU 的速度为: "+cpu.getSpeed());
        System.out.println("硬盘容量为: "+hd.getAmount());
    }
}

class Test{
    public static void main(String args[]){

```

```
CPU c=new CPU(2000);

HardDisk disk=new HardDisk(200);

PC pc=new PC();

pc.setHD(disk);

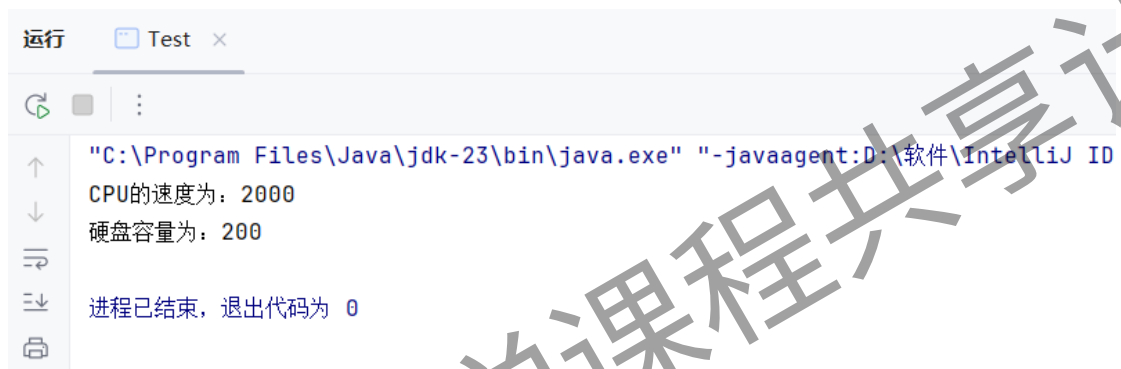
pc.setCPU(c);

pc.show();

}

}
```

【试验结果】



【结果分析】

程序运行结果符合预期，成功创建了 CPU 和 HardDisk 对象，并通过 PC 类的 setCPU 和 setHD 方法将它们与 PC 对象关联。调用 show 方法时，正确显示了 CPU 的速度和硬盘的容量。整个程序结构清晰，类之间的关系明确，实现了题目要求的功能，同时也展示了类的封装、组合和对象的使用，达到了设计目标。

1.3

实验内容:

3、构造日期类 `MyDate` 类，包含年月日，提供相应的 `get` 和 `set` 函数，设置日期时能实现日期合法性判断，包括大小月和闰年。可以利用 `MyDate` 类的 `toString()` 方法来实现类对象属性的展示，以 “****/**/” 的格式显示日期，例：2022/8/29。

【设计思路】

为了构造一个能够正确表示日期的 `MyDate` 类，我定义了三个私有成员变量 `day`、`month` 和 `year` 来存储日期的年、月、日信息。通过提供 `get` 和 `set` 方法，实现了对这些属性的访问和修改。在 `set` 方法中加入了合法性判断，确保输入的日期符合实际的日期规则，例如月份范围为 1 到 12，日期范围根据大小月以及闰年来判断。通过重写 `toString()` 方法，以“年/月/日”的格式展示日期信息，使得日期的输出更加直观和规范。主函数中通过用户输入创建 `MyDate` 对象并调用 `toString()` 方法展示日期，完成了对日期类的基本操作和展示功能。

【程序源代码】

```
import java.util.*;

public class MyDate {
    private int day, month, year;

    public MyDate() {}

    public MyDate(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
    }

    public int getDay() {
        return day;
    }
}
```

```
public void setDay(int day) {  
    this.day=day;  
    if(day>31||day<1)  
        System.out.println("输入无效");  
}
```

```
public int getMonth() {  
    return month;  
}
```

```
public void setMonth(int month) {  
    this.month=month;  
    if(month>12||month<1)  
        System.out.println("输入无效");  
}
```

```
public int getYear() {  
    return year;  
}
```

```
public void setYear(int year) {  
    this.year=year;  
    if(year<0)  
        System.out.println("输入无效");  
}
```

```
public String toString(){  
    return year+"/"+month+"/"+day;  
}
```

```
public static void main(String[] args) {
```

```

        Scanner scan=new Scanner(System.in);

        int year=scan.nextInt();

        int month=scan.nextInt();

        int day=scan.nextInt();

        MyDate d=new MyDate(day,month,year);

        System.out.println(d.toString());

    }
}

```

【试验结果】



【结果分析】

程序能够正确接收用户输入的年、月、日信息，并通过 `MyDate` 类的构造方法和 `toString()` 方法以指定格式输出日期。然而，当前的合法性判断仅对月份和日期的简单范围进行了检查，没有完全实现大小月和闰年的判断逻辑。例如，对于 2 月的日期范围（平年 28 天，闰年 29 天）以及 4、6、9、11 月的 30 天限制尚未完全实现，导致部分非法日期可能未被正确拦截。后续可以通过完善合法性判断逻辑，进一步提升日期类的准确性和实用性。

1.4

实验内容:

4、实现 Person 类及其子类 Student 类。

(1) 声明 Person 类，包含 name (String 类型)、age(int 类型)、sex (char 类型) 成员变量，通过构造方法进行赋值 (含构造方法重载)。一个 show 方法，

返回类型为 String 类型，返回内容为：姓名 性别 年龄。

(2) 声明 Student 类，继承 Person 类，增加学号 id (int 类型) 成员变量，通过构造方法，利用 super 调用父类构造方法实现成员变量赋值。Override 父类的 show 方法，返回 String 类型，返回内容为：姓名 性别 年龄 学号。

【设计思路】

我首先设计了 Person 类及其子类 Student 类，以实现对人员信息的基本管理和扩展。Person 类包含姓名、年龄和性别三个成员变量，并通过构造方法实现成员变量的初始化，同时提供 show 方法用于展示基本信息。Student 类继承自 Person 类，新增了学号成员变量，并通过构造方法调用父类构造方法完成初始化，同时覆盖了父类的 show 方法以扩展信息展示内容。Main 类通过创建 Person 和 Student 对象并调用其 show 方法，验证了类的继承和方法重写功能。

【程序源代码】

```
public class Person {  
    protected String name;//记得让子类能用  
    protected int age;  
    protected char sex;  
    Person(){}  
    Person(String name,int age,char sex){  
        this.name=name;  
        this.age=age;  
        this.sex=sex;  
    }  
}
```

```

    public String show(){
        return "姓名: "+name+" 年龄: "+age+" 性别: "+sex;
    }
}

class Student extends Person{
    private int id;
    Student(String name,int age,char sex,int id){
        super(name,age,sex);
        this.id=id;
    }
    public String show(){
        return "姓名: "+name+" 年龄: "+age+" 性别: "+sex+" 学号: "+id;
    }
}

class Main{
    public static void main(String[] args) {
        Person p=new Person("jennie",20,'女');
        System.out.println(p.show());
        Student s=new Student("jisoo",21,'女',1001);
        System.out.println(s.show());
    }
}

```

【试验结果】



```
运行 Main x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community E
姓名: jennie 年龄: 20 性别: 女
姓名: jisoo 年龄: 21 性别: 女 学号: 1001
进程已结束, 退出代码为 0
```

【结果分析】

程序运行结果符合预期, **Person** 类和 **Student** 类的构造方法正确初始化了成员变量, **show** 方法能够正确返回格式化的字符串信息。**Student** 类通过 **super** 关键字成功调用了父类构造方法, 实现了对父类成员变量的赋值, 并通过重写 **show** 方法扩展了信息展示内容。主函数中创建的 **Person** 和 **Student** 对象分别输出了正确的信息, 验证了类的继承、构造方法的重载和方法的重写功能, 实现了题目要求的功能。

1.5

实验内容：

定义接口 Shape 及其抽象方法 getArea()和 getPerimeter()用于计算图形和面积和周长。定义类 Rectangle(矩形)、类 Circle(圆形)、类 Triangle(三角形)，要求这些类继承点类 Coordinates()并实现接口的抽象方法。

【设计思路】

首先定义一个 Shape 接口来规范图形类的行为，接口中包含计算面积和周长的抽象方法 getArea() 和 getPerimeter()。为了表示图形的位置，定义了 Coordinate 类来存储二维坐标。随后，定义了 Rectangle（矩形）、Circle（圆形）和 Triangle（三角形）三个类，它们都继承自 Coordinate 类并实现 Shape 接口。每个类根据其几何特性实现了面积和周长的计算逻辑。主类 Main_Shape 中创建了三类图形对象，并调用它们的方法来展示面积和周长的计算结果，验证了接口的多态性和抽象方法的实现。

【程序源代码】

```
public interface Shape {  
    double getArea();//默认 public abstract，抽象类中可以写 abstract 或者不写  
    double getPerimeter();  
}  
  
class Coordinate  
{ int x, y;  
    Coordinate(int x, int y)  
    {  
        this.x=x;  
        this.y=y;  
    }  
}
```

```
class Rectangle extends Coordinate implements Shape{
    Coordinate c;
    private double width, height;
    Rectangle(int x, int y, double width, double height){
        super(x,y);
        this.width=width;
        this.height=height;
    }

    public double getArea(){
        return width*height;
    }
    public double getPerimeter(){
        return 2*(width+height);
    }
}
```

```
class Circle extends Coordinate implements Shape{
    Coordinate c;
    private double radius;
    Circle(int x, int y, double radius){
        super(x,y);
        this.radius=radius;
    }

    public double getArea(){
        return Math.PI*radius*radius;
    }
}
```

```

        public double getPerimeter(){
            return 2*Math.PI*radius;
        }
    }

class Triangle extends Coordinate implements Shape{
    Coordinate c;
    private double a,b,d;
    Triangle(int x, int y, int a,int b,int d){
        super(x,y);
        this.a=a;
        this.b=b;
        this.d=d;
    }

    public double getArea(){
        double s=(getPerimeter()/2);
        return Math.sqrt(s*(s-a)*(s-b)*(s-d));
    }
    public double getPerimeter(){
        return a+b+d;
    }
}

class Main_Shape{
    public static void main(String[] args) {
        Shape s1 = new Rectangle(1,1,2,3);
        Shape s2 = new Circle(1,1,2);
        Shape s3 = new Triangle(1,1,2,3,4);
    }
}

```

```
        System.out.println(s1.getArea());

        System.out.println(s2.getArea());

        System.out.println(s3.getArea());

        System.out.println(s1.getPerimeter());

    }

}
```

【试验结果】



```
运行 Main_Shape x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Communit
6.0
12.566370614359172
2.9047375096555625
10.0
进程已结束, 退出代码为 0
```

【结果分析】

程序运行结果符合预期，成功创建了矩形、圆形和三角形对象，并正确计算了它们的面积和周长。Rectangle 类通过宽度和高度计算面积和周长；Circle 类通过半径计算面积和周长；Triangle 类通过三边长度使用海伦公式计算面积和直接求和计算周长。主类中通过接口类型的引用调用具体类的方法，体现了多态性。程序结构清晰，接口和类的设计合理，能够满足不同图形的面积和周长计算需求。